



Donde la localidad es costumbre

MEMORIA TÉCNICA · SISTEMA DE RECOMENDACIÓN

Data Science — Asistente Cultural y Gastronómico del País Vasco

Desafío de Tripulaciones · Junio 2026

Contenido

#	Sección
1	Introducción y Contexto
2	Fuentes de Datos
3	Procesamiento y Limpieza
4	Análisis Exploratorio de Datos
5	Arquitectura de Recomendación
6	Modelos de Recomendación
7	Evaluación y Validación
8	Integración (Chatbot, API y Backend)
9	Roadmap Futuro

1. Introducción y Contexto

SustraiApp es una plataforma de recomendación cultural y gastronómica diseñada específicamente para el público del País Vasco. El objetivo es convertir peticiones en lenguaje natural en recomendaciones de alta calidad que combinen experiencias gastronómicas, patrimonio cultural y eventos de interés.

El sistema de recomendación está orientado a un público objetivo de 45–65 años que valora experiencias premium pero no elitistas. Estos usuarios buscan calidad, autenticidad y relevancia local, con una incidencia alta de preferencia por búsquedas en lenguaje natural (chatbot) frente a filtros tradicionales.

Esta memoria documenta el pipeline completo de Data Science: desde las fuentes de datos hasta la arquitectura final de seis modelos de recomendación (dos por categoría de contenido), con mecanismos específicos para usuarios en fase cold-start y usuarios con histórico de interacciones.

2. Fuentes de Datos

El catálogo de recomendación se construye a partir de múltiples fuentes oficiales y de terceros:

Fuente	Contenido	Registros
Open Data Euskadi	Gastronomía, Patrimonio	~1300
Kulturklik	Eventos culturales, conciertos	~2800
Google Places	Valoraciones, reseñas, precios	~900
Listados Oficiales	Michelin, Repsol, Denominaciones	~150

3. Procesamiento y Limpieza de Datos

3.1 Criterios de Filtrado

Los datos de catálogo se sometieron a criterios de calidad estrictos:

GASTRONOMÍA

Valoración > 4.0 · Rango de precios moderado–muy caro · Negocios establecidos.

PATRIMONIO CULTURAL

Valoración > 4.0 · Contenido educativo/experiencial · Sitios de calidad.

EVENTOS

Filtrado por fecha · Verificación de datos · Eventos históricos para variedad.

3.2 Estandarización de Taxonomía

Se definió una taxonomía unificada de 28 categorías que agrupa los tres tipos de contenido (gastronomía, patrimonio, eventos). Cada ítem recibe un identificador numérico (id_interes) que lo mapea a su categoría en esta taxonomía global.

Los sellos de calidad (Denominación de Origen, Eusko Label, Euskal Baserri, Agricultura Ecológica, Michelin, Repsol) se codifican como características adicionales para enriquecer los perfiles de los ítems.

3.3 Validación de Datos

Se implementó un script de validación (validar_datos.py) que ejecuta después de cada fase de limpieza. Este script comprueba:

- ✓Integridad referencial: ningún ítem con provincia o categoría nula o inválida.
- ✓Duplicados: no hay item_id repetidos dentro de un CSV o entre CSV (usando prefijos: g-, p-, e-).
- ✓Rango de valoraciones: todas entre 1–5; se imputan los nulos a la media histórica (4.3).

3.4 Persistencia en Base de Datos Compartida

Tras la limpieza y validación, los catálogos dejan de gestionarse únicamente como ficheros CSV y se vuelcan a la base de datos PostgreSQL compartida con el equipo de Full Stack, que pasa a ser la fuente de verdad del proyecto. El chatbot puede seguir leyendo de los CSV como snapshot resiliente (`DATA_SOURCE = csv`) o directamente de la base compartida (`DATA_SOURCE = db`), sin cambiar el código.

Se incorporó además la tabla de los 252 municipios del País Vasco con sus códigos INE oficiales (como `nora_code`), el código de provincia y las coordenadas de centroide del IGN. El campo `nora_code` se mantiene como texto para preservar los ceros a la izquierda de los códigos de Araba. El detalle de esta integración se describe en la Sección 8.4.

4. Análisis Exploratorio de Datos (EDA)

4.1 Distribución por Provincia

El catálogo está bien distribuido entre las tres provincias: Bizkaia (~45%), Gipuzkoa (~35%), Araba (~20%). Esta distribución refleja la densidad real de oferta cultural y gastronómica del País Vasco.

4.2 Distribución Temática

Las categorías más representadas son Restaurantes, Eventos Culturales Variados, Bodegas y Museos. Las categorías menos comunes (Bertsolarismo, Concursos) tienen presencia pero limitada, lo que requiere especial cuidado en la evaluación de cold-start.

4.3 Valoración

La distribución de valoraciones es sesgada hacia calificaciones altas (media: 4.3 ★). Esto es consistente con el filtrado previo (> 4.0) y refleja la calidad del catálogo; sin embargo, reduce la varianza en el espacio de características para la recomendación.

5. Arquitectura de Recomendación

El sistema combina dos enfoques complementarios por cada categoría de contenido:

FILTRADO COLABORATIVO (SVD)

Cuando el usuario ya tiene histórico de reseñas, se factoriza la matriz usuario $\times$ ítem para capturar patrones latentes de gusto compartidos entre usuarios similares.

FILTRADO POR CONTENIDO (GRADIENTBOOSTING)

Basado en características explícitas del ítem (categoría, provincia, sellos) y del usuario (preferencias, ubicación, afinidad extendida). Cubre el cold-start y los ítems nuevos sin reseñas.

La puntuación final es una combinación lineal ponderada:

$$\text{score} = \alpha \times \text{SVD} + (1 - \alpha) \times \text{Contenido}$$

El peso α se selecciona empíricamente por categoría barriendo valores de 0 a 1 y midiendo el RMSE sobre

un conjunto de validación común. El valor óptimo varía según la densidad de interacciones disponibles en cada categoría (ver Sección 7).

6. Modelos de Recomendación

El sistema implementa SEIS MODELOS: dos componentes por cada categoría de contenido (Gastronomía, Patrimonio, Eventos). Cada categoría combina un componente colaborativo (SVD) para usuarios con histórico y un componente de contenido (GradientBoosting) para el cold-start.

6.1 Componente colaborativo — SVD

El componente colaborativo se implementa con la librería scikit-surprise, empleando el algoritmo SVD (factorización matricial tipo Funk SVD). La entrada son tripletas (user_id, item_id, puntuación) con escala de valoración 1–5 (Reader rating_scale=(1,5)).

Configuración común a las tres categorías:

- Factores latentes: n_factors = 50.
- Épocas de entrenamiento: n_epochs = 30.
- Semilla fija: random_state = 42 (reproducibilidad).
- Selección de algoritmo mediante validación cruzada 5-fold (RMSE y MAE) frente a BaselineOnly, KNNWithMeans y NMF.
- Entrenamiento final sobre el dataset completo (build_full_trainset) para el modelo de producción.

Es el modelo principal cuando el usuario ya ha generado reseñas dentro de la aplicación, puesto que captura la señal colaborativa de usuarios con gustos similares.

6.2 Componente de contenido y cold-start — GradientBoosting

Para usuarios nuevos sin histórico (y para ítems recién publicados sin reseñas, como eventos futuros) se usa un GradientBoostingRegressor de scikit-learn dentro de un Pipeline con imputación por mediana (SimpleImputer). Este componente se apoya en el feature engineering del proyecto:

- Intereses del usuario procedentes del formulario de onboarding (vector de intereses).
- Características one-hot del ítem: tipo de lugar, provincia, sellos de calidad.
- Afinidad extendida: producto interés del usuario × tipo de ítem.

En eventos se incorporan además señales temporales y de precio genuinamente independientes: month, weekday, is_weekend, is_free, online y price_eur transformado con log1p y recorte (clipping) al percentil 95 para neutralizar outliers extremos de precio.

6.3 Combinación híbrida

Ambos componentes se combinan linealmente con $\text{score} = \alpha \times \text{SVD} + (1 - \alpha) \times \text{Contenido}$. El α óptimo se busca empíricamente por categoría. En patrimonio el barrido converge a $\alpha = 1.0$ (SVD puro), lo que indica que, con la densidad de interacciones disponible, el componente colaborativo ya captura la totalidad de la señal aprendible; el componente de contenido se mantiene de todos modos por su valor en cold-start y explicabilidad.

6.4 Mapa de los seis modelos

Categoría	Caso de uso	Técnica
 Gastronomía	Usuario nuevo (cold-start)	GradientBoosting (contenido)

Categoría	Caso de uso	Técnica
📄 Gastronomía	Usuario con histórico	SVD (colaborativo)
📄 Patrimonio	Usuario nuevo (cold-start)	GradientBoosting (contenido)
📄 Patrimonio	Usuario con histórico	SVD (colaborativo)
📄 Eventos	Usuario nuevo / evento sin reseñas	GradientBoosting (contenido)
📄 Eventos	Usuario con histórico	SVD (colaborativo)

NOTA SOBRE EVENTOS

Al igual que en gastronomía y patrimonio, los datos de este bloque se han generado de forma sintética en `reviews_event.csv` (83.485 reseñas, 2.000 usuarios × 2.744 eventos, densidad ≈ 1,5%). Sin embargo, conceptualmente representan un histórico de eventos pasados ya finalizados. Esta distinción es clave para el filtrado colaborativo: permite modelar las preferencias de los usuarios sobre experiencias anteriores y, mediante técnicas de similitud, predecir la valoración o aceptación de los nuevos eventos del catálogo antes de que se celebren.

7. Evaluación y Validación

7.1 Metodología

La evaluación se estructura en dos fases. Primero, una selección de algoritmo mediante validación cruzada 5-fold que compara los candidatos colaborativos (BaselineOnly, KNNWithMeans, SVD, NMF). Segundo, una comparación SVD vs. Contenido sobre un mismo split 80/20 (train/test) para garantizar una evaluación justa entre ambos componentes. La métrica principal es el RMSE, complementada por el MAE.

7.2 Selección de algoritmo (validación cruzada 5-fold)

En las tres categorías, SVD es el mejor algoritmo colaborativo del conjunto evaluado. RMSE medio (menor es mejor):

Algoritmo	Gastronomía	Eventos
BaselineOnly	1.0200	1.1467
KNNWithMeans	1.1317	1.3609
SVD ✓	1.0490	1.1734
NMF	1.2349	1.2876

En patrimonio, SVD obtiene en validación cruzada 5-fold un RMSE de 0.8399 ± 0.0148 y un MAE de 0.6444, confirmando el mismo patrón.

7.3 Rendimiento del modelo final (test hold-out 80/20)

Comparación de los dos componentes sobre el conjunto de test, por categoría (RMSE / MAE):

Categoría	SVD (RMSE / MAE)	Contenido (RMSE / MAE)
📄 Patrimonio	0.6017 / 0.4602	0.8441 / 0.6231
📄 Gastronomía	0.7595 / 0.5861	0.9900 / 0.7456
📄 Eventos	0.8210 / 0.6445	1.1250 / 0.8401

7.4 Análisis de resultados

- SVD supera de forma consistente al componente de contenido en RMSE y MAE en las tres categorías, por lo que es el modelo principal siempre que el usuario tenga histórico.
- El componente de contenido (GradientBoosting) presenta un RMSE mayor, pero es imprescindible: es el único capaz de recomendar a usuarios nuevos y a ítems sin reseñas (especialmente eventos).

futuros).

- Suelo de ruido en eventos: la desviación estándar natural de las puntuaciones ($\text{Std} \approx 1.13$) marca el límite inferior alcanzable. Con un RMSE de SVD ≈ 0.82 , el modelo ya opera por debajo de esa cota, señal de que extrae prácticamente toda la señal aprendible; el margen restante está dominado por el ruido intrínseco de los datos, no por la elección de hiperparámetros.
- El α óptimo = 1.0 en patrimonio confirma que, con densidad de interacciones suficiente, el colaborativo domina; en categorías más dispersas el componente de contenido aporta mayor valor relativo.

8. Integración: Chatbot, API de Recomendación y Backend Full Stack

El sistema de recomendación no funciona de forma aislada: se ha desplegado como un conjunto de servicios coordinados que comparten una misma base de datos con el equipo de Full Stack. Esta sección documenta la arquitectura de integración resultante —la API que expone los modelos, el servicio de chatbot conversacional y el volcado de los catálogos a la base compartida— así como el trabajo de coordinación de endpoints y campos que garantiza la compatibilidad entre verticales.

8.1 Arquitectura de integración (topología de servicios)

La solución se articula en tres servicios independientes, cada uno con una responsabilidad clara, que conversan entre sí mediante API REST y comparten una única base de datos PostgreSQL como fuente de verdad:

Servicio	Responsable	Rol
Backend principal (Flask · :5000)	Full Stack	Autenticación, perfil de usuario, preferencias, intereses, reseñas, municipios y eventos. Persiste en PostgreSQL.
API de recomendación (recsys)	Data Science	Expone los seis modelos entrenados como endpoints REST y devuelve recomendaciones de lugares y eventos con puntuación calibrada (1–5).
Orquestador / chatbot (Flask · :5001)	Data Science	Traduce el lenguaje natural, llama a la API de recomendación, personaliza con datos del backend y redacta la respuesta para el cliente.

FLUJO DE DATOS

Cliente → Chatbot (Flask): el LLM traduce la petición → API de recomendación (modelos): calcula la predicción → Chatbot: el LLM redacta la respuesta → Cliente. En paralelo, el chatbot consulta al backend de Full Stack los intereses y preferencias del usuario para personalizar la recomendación. Todos los servicios leen del mismo PostgreSQL.

8.2 API de Recomendación: exponer los modelos como servicio

Para que los modelos dejaran de ser notebooks y pasaran a ser consumibles por el resto del equipo, se ha levantado una API REST que los envuelve tras un contrato estable. El chatbot —y, en general, cualquier cliente— obtiene recomendaciones por HTTP sin conocer los detalles internos de los modelos. Los endpoints principales son:

- **GET /recommendations/places?**provincia=&intereses=&top_n= — recomienda lugares fijos (gastronomía, patrimonio).
- **GET /recommendations/events?**provincia=&intereses=&fecha_inicio=&fecha_fin=&top_n= — recomienda eventos en un rango de fechas.
- **GET /taxonomy** — devuelve el catálogo completo de las 29 categorías de interés (gastronomía,

patrimonio y eventos) con sus identificadores numéricos, de modo que el chatbot, el frontend y el backend puedan validar y traducir intereses dinámicamente sin hardcodear la lista en su código.

- **GET /health** — devuelve el estado operativo de la API de recomendación. Lo usan el orquestador del chatbot y el backend para verificar que el servicio está levantado antes de lanzar llamadas, y para detectar caídas de forma temprana en lugar de recibir errores silenciosos.

Todas las respuestas comparten un mismo envelope por ítem: `item_id`, nombre, tipo (lugar | evento), categoría, provincia, municipio, `estrella_prevista` (escala común 1–5) y url. Este formato único es el que el chatbot espera y el que cualquier consumidor recibe, sea cual sea la categoría.

DISEÑO POR CONTRATO (CONTRACT-FIRST)

Se definió primero el envelope de respuesta y los endpoints, y se entregó un stub que los cumplía exactamente. Así, Full Stack pudo integrar contra la API antes de que los modelos reales estuvieran desplegados. El paso a producción consiste únicamente en apuntar la variable `MODEL_API_URL` a la API real: no cambia ni una línea del orquestador ni del contrato. Además, el chatbot opera en modo HTTP estricto — si la API no responde, avisa de forma explícita en lugar de degradar en silencio —, lo que facilita depurar la integración entre equipos.

8.3 Chatbot conversacional: lenguaje natural en ambos sentidos

El chatbot es el punto de entrada conversacional de SustraiApp. Incorpora un modelo de lenguaje ejecutado en local mediante Ollama (llama3.1:8b) que actúa como traductor bidireccional entre el usuario y los modelos de recomendación: convierte la petición en lenguaje natural en una llamada estructurada y, después, convierte la predicción de los modelos de nuevo en lenguaje natural. El flujo completo de un turno es:

#	Paso
1	Traducir (entrada): el LLM convierte la petición del usuario en una llamada de herramienta (function calling) con provincia, intereses (ids de taxonomía), fechas e intención.
2	Grounding: el servidor revalida y corrige contra la taxonomía y los gazetteers. Las fechas se calculan en el servidor, nunca las inventa el LLM, y se descartan las categorías inexistentes.
3	Recomendar: se llama a la API de recomendación con los parámetros ya validados y se recoge la predicción de los modelos.
4	Redactar (salida): el LLM convierte los ítems devueltos en una sugerencia breve y concreta en lenguaje natural, usando solo los resultados reales.
5	Responder: JSON con el texto, los ítems estructurados y los metadatos de la consulta.

El servicio incorpora además una memoria de sesión que recuerda la provincia y los intereses del turno anterior (sin almacenar texto ni datos personales), de modo que un seguimiento como «¿y mañana?» o «¿y en Donosti?» no pierde el contexto. Si la petición incluye un `user_id`, el chatbot hereda los intereses guardados del usuario en el backend de Full Stack para personalizar la respuesta.

DOBLE TRADUCCIÓN, DECISIÓN EN LOS MODELOS

El LLM nunca decide la recomendación: traduce la petición a parámetros y, después, traduce los resultados a lenguaje natural. La recomendación la calculan siempre los modelos, y el grounding en servidor evita que el LLM invente categorías, provincias o fechas. Al ejecutarse el LLM en local, no hay coste por token ni se envían datos del usuario a terceros.

8.4 Base de datos compartida: volcado de los catálogos a PostgreSQL

Los catálogos de Data Science (gastronomía, patrimonio y eventos), que durante el desarrollo vivían como

ficheros CSV, se han volcado a la base de datos PostgreSQL que mantiene el equipo de Full Stack. Con ello se elimina la duplicación de datos: ambos equipos leen de la misma base, que pasa a ser la única fuente de verdad del proyecto.

- **Doble origen configurable:** el chatbot puede leer tanto de los CSV (snapshot resiliente) como de la base compartida, sin cambiar el código, según la variable `DATA_SOURCE = csv | db`.
- **Carga de municipios:** se cargaron los 252 municipios del País Vasco con sus códigos INE oficiales (como `nora_code`), el código de provincia y las coordenadas de centroide del IGN.
- **Detalle crítico de tipos:** el código `nora_code` se mantiene como texto para preservar los ceros a la izquierda de los códigos de Araba, que un tipo entero destruiría.
- **Validación previa:** el script `validar_datos.py` comprueba integridad referencial, ausencia de duplicados y rango de valoraciones antes de la carga.

DISCIPLINA DE DATOS PERSONALES

Solo los catálogos de contenido público se vuelcan a la base compartida. Los ficheros con información personal (correos, teléfonos, hashes de contraseña) ni se versionan en Git ni se cargan, manteniendo la separación entre datos de contenido y datos sensibles de usuario.

8.5 Coordinación de contratos y campos con Full Stack

Que tres servicios desarrollados por verticales distintas funcionen sobre una misma base sin fricciones exigió un trabajo explícito de coordinación de contratos y campos. Los acuerdos principales con el equipo de Full Stack fueron:

- **Diseño conjunto de endpoints:** se definieron conjuntamente rutas, parámetros y códigos de estado (documentados en `RUTAS_API.md`), de forma que el chatbot pudiera consumir el backend —intereses, preferencias, municipios, eventos— y que el backend pudiera consumir la API de recomendación.
- **Envelope de respuesta unificado:** el backend envuelve sus respuestas en una clave «data»; el cliente del chatbot la desenvuelve de forma transparente, evitando errores de parseo entre equipos.
- **Alineación de taxonomías:** el backend maneja una taxonomía de 17 categorías y el chatbot/recsys una de 29. Se acordó una capa de traducción por nombre (`MAPA_INTERES_BACKEND`) que convierte los ids de un sistema al otro, de modo que un interés guardado por el usuario se traduce correctamente a los ids que entienden los modelos.
- **Campos compartidos:** provincia normalizada (Araba / Bizkaia / Gipuzkoa), municipios con código INE común y escala de valoración 1–5 compartida en todos los servicios.
- **Resiliencia entre servicios:** todas las llamadas entre servicios son tolerantes a fallos. Si un servicio no responde, el resto sigue funcionando en modo degradado controlado en lugar de caer.

RESULTADO

Una única base de datos, una taxonomía conciliada y contratos de API estables. Esto elimina los problemas de compatibilidad típicos entre verticales —campos con nombres distintos, ids que no casan, formatos de respuesta divergentes— y permite que Front, Back y Data Science evolucionen en paralelo sin romperse mutuamente.

9. Roadmap Futuro

9.1 Mejoras de Datos

- Integración de datos de accesibilidad para usuarios con movilidad reducida.

- Información detallada sobre alergias e intolerancias en restaurantes.
- Enriquecimiento de eventos con transporte público y aparcamiento.

9.2 Expansión de Contenido

- Hoteles rurales y alojamientos de calidad.
- Visitas guiadas temáticas.
- Experiencias personalizadas (catas, workshops, clases de cocina).

9.3 Mejoras de Modelo

- Recalibración de α con datos reales de usuario.
- Incorporación de contexto temporal (estacionalidad, patrones por día de la semana).
- Bandits multibrazo para exploración/explotación.
- Transición a la API de modelos en producción: el contrato de la API de recomendación ya está validado de extremo a extremo contra un stub equivalente, por lo que el paso de `MODEL_API_MODE = mock` a `http` consiste únicamente en apuntar el orquestador a la API que sirve los modelos reales, sin cambios en el chatbot.

Two-Tower: la arquitectura objetivo para la parte dinámica de eventos

Los eventos son contenido inherentemente dinámico: aparecen y caducan constantemente, y los más relevantes suelen ser justo los que todavía no tienen reseñas. El enfoque actual (SVD colaborativo + GradientBoosting para cold-start) cubre ese hueco con dos modelos separados; la arquitectura objetivo en producción es un modelo único de tipo Two-Tower.

Cómo funciona. Son dos redes neuronales (“torres”) entrenadas a la vez. Una codifica al usuario (intereses, demografía, histórico) y otra codifica al evento (tipo, precio, mes, día de la semana, ubicación); cada una produce un vector (embedding) en un mismo espacio. La afinidad usuario–evento es el producto escalar entre ambos vectores. Los embeddings de eventos se precálculan y se indexan (búsqueda por vecinos aproximados, p. ej. FAISS), de modo que recuperar los mejores eventos para un usuario es casi instantáneo aunque haya miles.

Diferencia con lo implementado. El SVD aprende factores latentes solo a partir de la matriz de reseñas: no puede puntuar un evento que no estaba en el entrenamiento, por eso hace falta el modelo de contenido aparte. El Two-Tower aprende de características, así que un evento nuevo —sin ninguna reseña— se puntúa al instante pasando sus features por la torre de eventos. Resuelve el cold-start de eventos de forma nativa y dentro de un único modelo, e integra de serie las señales temporales y de precio que hoy quedan infrutilizadas.

Cómo mejoraría en producción. Recomendaría eventos desde el momento en que se publican (sin esperar a acumular reseñas), unificaría el scoring en un solo modelo (eliminando el ajuste de α y el blend de dos modelos), escalaría a catálogos grandes gracias al índice vectorial y permitiría reentrenamiento incremental a medida que llegan nuevas interacciones.

9.4 Observabilidad

- Recolección sistemática de feedback implícito (clics, dwell time, ratings).
- Dashboards de performance por categoría y provincia.
- Framework de A/B testing para validar mejoras en producción.



Donde la localidad es costumbre